

Outlines

- Introduction
- Types of polymorphism
- Binding - static & Dynamic
- Memory Allocation
- Pointer to object
- Pointers to derived classes
- Virtual Functions
- Pure Virtual functions
- Abstract Class

Polymorphism

Poly = many

Morph = forms

Polymorphism: Ability to take more than one form. Processing objects differently based on their data type.

Polymorphism allows functions or methods to behave differently based on the object that is calling or invoking them.

Examples:

Function overloading, Operator overloading, Constructor overloading, Function overriding

Types of Polymorphism

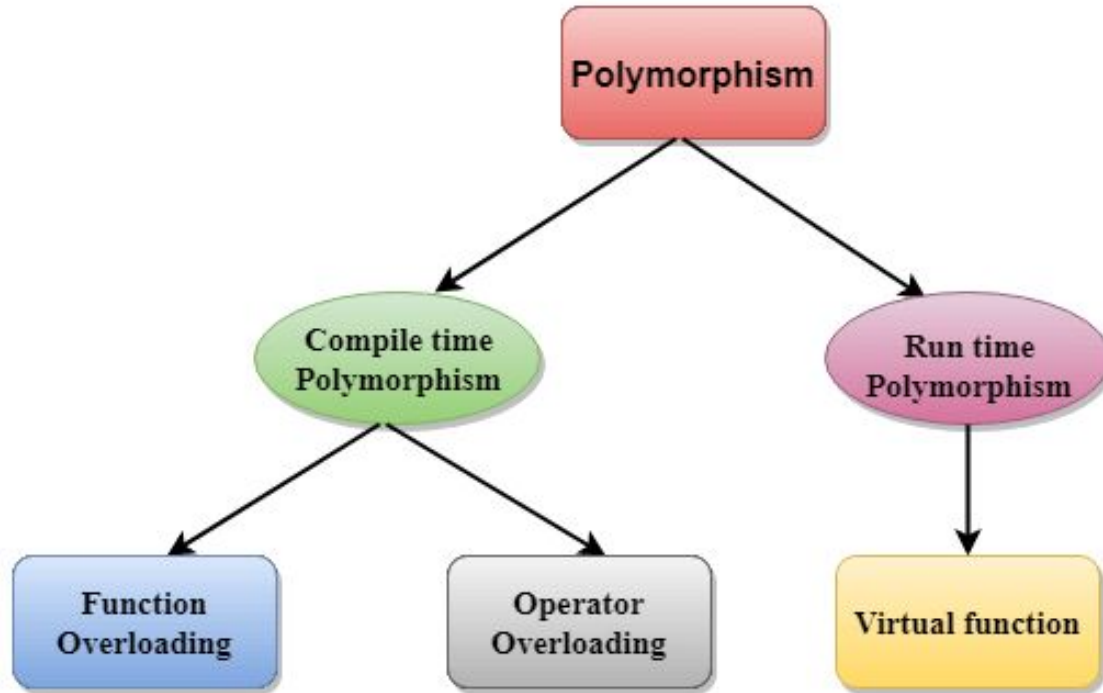
Compile-time (Static) Polymorphism: refers to forms of polymorphism that are resolved by the compiler at compile time.

- It's also known as early binding or static binding.
- The function to be invoked is determined at compile time.
- It is achieved using function overloading and operator overloading

Run-time (Dynamic) Polymorphism: refers to forms of polymorphism that are resolved at runtime.

- It is also known as late binding or dynamic binding.
- It is achieved using function/method overriding (through virtual functions)
- The function to be invoked is determined at runtime based on the object type that invokes the function.

Types of Polymorphism



Compile Time Polymorphism

In Compile time Polymorphism, the function call is resolved at the compile time.

It is also known as Static binding or Early binding.

It is achieved by function/method overloading or operator overloading

It provides fast execution because the method that needs to be executed is known early at the compile time.

Compile time polymorphism is less flexible as all things execute at compile time.

Inheritance is not involved.

Run time Polymorphism

In Run time Polymorphism, the function call is resolved at run time.

It is also known as Dynamic binding or Late binding

It is achieved by function overriding (using virtual functions and pointers).

It provides slow execution as compare to early binding because the method that needs to be executed is known at the runtime.

Run time polymorphism is more flexible as all things execute at run time.

Inheritance is involved.

Example of Compile-Time Polymorphism using function overloading

```
#include <iostream>
using namespace std;

class MyClass {
public:
    void display(int i) {
        cout << "Integer: " << i << endl;
    }

    void display(double f) {
        cout << "Float: " << f << endl;
    }

    void display(string s) {
        cout << "String: " << s << endl;
    }
};
```

```
int main() {
    MyClass m;
    m.display(10);           // Calls display(int)
    m.display(3.14);         // Calls display(double)
    m.display("Hello");      // Calls display(string)
    return 0;
}
```

Binding

Binding refers to the process of associating a function call with the function definition.

Types:

Static (Compile-time) Binding: Static binding occurs when the function to be invoked is determined at compile time (i.e. the compiler knows at the compile-time which function is called). It is faster but less flexible.

Dynamic (Run-time) Binding: Dynamic binding occurs when the function to be invoked is determined at runtime (i.e. a compiler does not know at compile-time which functions to call up until the run-time). It is slower but more flexible.

Recall Pointers

- A variable that stores the memory address as its value. It is created with the * operator.
- Pointers are said to "point to" the variable whose address they store.
- Pointers can be used to directly access the variable they point to using the dereference operator (*).

```
int a = 5;
int* ptr = &a; // Assign the address of a to the pointer
*ptr = 10; // a's value will now be 10
int b = *ptr; // A new variable b will be created and a's value will be copied to b
```

Array of pointers:

```
data_type *var_name[array_size];
int *price[3];
```


Memory Allocation

Memory allocation is the process of reserving a portion of memory for program use. Memory allocation is a fundamental aspect of programming that allows programs to store and manipulate data.

Types:

Static Memory Allocation: Memory is allocated at compile time and has a fixed size.

- Done by the compiler automatically (implicitly).
- Allocation and free are done implicitly.
- No need to explicitly manage memory.

Dynamic Memory Allocation: Memory is allocated at runtime and has flexibility in memory usage.

- Done explicitly by the programmer. Programmer explicitly requests the system to allocate memory and return starting address of memory allocated.
- When done using memory, it must be explicitly freed.
- It is essential to manage dynamic memory carefully to avoid leaks and other memory-related issues.

Dynamic memory is allocated using operator **new** followed by a data type specifier.

```
int *ptr = new int;
```

Once the memory allocated using **new** operator, it can be freed using operator **delete** so that the memory becomes available.

```
delete ptr;
```

The dot (.) and arrow (->) operator

The dot operator (.) is used to access members of an object directly.

The arrow operator (->) is used to access members of an object through a pointer.

```
class Person {  
public:  
    string name;  
    void display() {  
        cout << "Name: " << name << endl;  
    }  
};
```

`ptrPerson->display();`

is equivalent to

`(*ptrPerson).display();`

```
int main() {  
    Person p; // Creating an object of Person  
    p.name = "Ram";  
    p.display();  
    return 0;  
}
```

```
int main() {  
    Person p; // Creating an object of Person  
    Person* ptrPerson = &p;  
    ptrPerson->name = "Ram";  
    ptrPerson->display();  
    return 0;  
}
```

Pointer to Object

A pointer to an object is a variable that stores the memory address of an object.

Explanation:

- Imagine you have a book (an object) and a bookmark (a pointer).
- The bookmark doesn't contain the book itself but tells you where the book is located.
- Similarly, a pointer to an object doesn't hold the object itself but holds the memory address where the object is stored.

Example to demonstrate how pointers to objects work using static memory allocation.

```
#include <iostream>
using namespace std;

class Person {
public:
    string name;
    void display() {
        cout << "Name: " << name << endl;
    }
};

int main() {
    Person p; // Creating an object of Person (static memory allocation)
    p.name = "Ram";

    Person* ptrPerson = &p; // Creating a pointer to the object
    ptrPerson->display(); // Accessing the object's member function using the pointer

    return 0;
}
```

Pointer to object using Dynamic Memory Allocation

A pointer to an object using the **new** operator allows you to dynamically allocate memory for an object at runtime.

Explanation:

- The **new** operator allocates memory for an object on the heap and returns a pointer to the allocated memory.
- This pointer can then be used to access the object's members and functions.

Example to demonstrate how pointers to objects work using dynamic memory allocation.

```
#include <iostream>
using namespace std;
class Person {
public:
    string name;
    void display() {
        cout << "Name: " << name << endl;
    }
};

int main() {
    Person* ptrPerson = new Person; // Dynamically creating an object of Person
    ptrPerson->name = "Ram"; // Setting the name of the Person
    ptrPerson->display(); // Accessing the object's member function using the pointer

    delete ptrPerson; // Deleting the dynamically allocated object to free memory
    return 0;
}
```

Pointers to Derived Classes

A pointer to a derived class is a pointer that holds the address of an object of the derived class.

This concept is essential for achieving **polymorphism** and **dynamic binding**, allowing a base class pointer to point to a derived class object.

Some rules to be followed:

Base Class Pointer:

- A base class pointer can point to an object of the base class or any derived class of that base class.
- By using base class pointer, we can access only those members which are inherited from the base class and not the members that originally belong to derived class.


```
class Base
{...};
class Derived : public Base
{...};
int main()
{
    Base b1, *p;
    Derived d1;
    p = &b1; // valid
    p = &d1; // also valid
}
```

Derived Class Pointer:

- A derived class pointer can only point to an object of the derived class but not of the base class or any of the derived classes of the base class.
- It can access members specific to the derived class that are not present in the base class.

Pointer to Derived classes

In case a member of derived class has the same name as one of the members of base class, then any reference to that member by pointer will always access the base class member.

```
class Base {
public:
    int value;
    void display() {
        cout << "Base class value: " << value << endl;
    }
};

class Derived : public Base {
public:
    int value; // Hides the base class member
    void display() {
        cout << "Derived class value: " << value <<
endl;
    }
};
```

```
int main() {  
    Base* basePtr;  
  
    Derived derivedObj;  
  
    basePtr = &derivedObj; // Base class pointer  
                             pointing to derived class object  
  
    basePtr->value = 10; // Accesses Base class value  
    derivedObj.value = 20; // Accesses Derived class  
                           value  
  
    basePtr->display(); // Calls Base class display  
                        function  
    derivedObj.display(); // Calls Derived class  
                          display function  
    return 0; }  

```

Virtual Functions

Virtual means existing in appearance but not in reality.

A virtual function is a member function in a base class that you can override in a derived class.

It is declared using the **virtual** keyword in the base class.

When a function is made **virtual**, C++ determines which function to use at run time based on the type of object pointed by the base pointer rather than the type of the pointer (Virtual functions use dynamic binding/late binding).

Purpose:

- Achieve runtime polymorphism.
- Allow derived classes to provide specific implementations of functions that are defined in the base class.

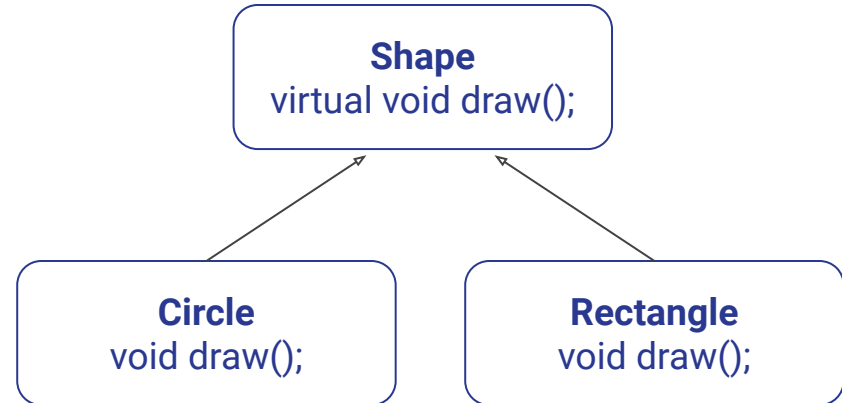
We must access virtual function through the use of pointer declared as a pointer to the base class.

Why can't we use the object name with dot operator the same way as any other member function to call the virtual function?

Yes, we can but run time polymorphism is achieved only when a virtual function is accessed through a pointer to the base class.

So, how runtime/dynamic polymorphism is actually achieved?

Let's consider an example with a base class **Shape** and derived classes **Circle** and **Rectangle**. The **draw** function is a virtual function.



Function Overriding (Recall)

Definition: When the base class and derived class have member functions with exactly the same name, same return-type, and same arguments list, then it is said to be function overriding.

Purpose: Allows a derived class to offer a specialized behavior for a method that is already provided by its base class.

Overriding is needed when derived class function has to do some added or different job than the base class function.

It is used to achieve runtime polymorphism.

Example program to achieve runtime polymorphism using virtual function

```
class Shape {
public:
    virtual void draw() {
        cout << "Drawing a Shape" << endl;
    }
};

class Circle : public Shape {
public:
    void draw() {
        cout << "Drawing a circle" << endl;
    }
};

class Rectangle : public Shape {
public:
    void draw() {
        cout << "Drawing a rectangle" << endl;
    }
};
```

```
int main() {
    Shape* ptrShape;
    Circle circle;
    Rectangle rectangle;

    ptrShape = &circle;
    ptrShape->draw(); // Calls Circle's draw
method

    ptrShape = &rectangle;
    ptrShape->draw(); // Calls Rectangle's
draw method

    return 0;
}
```

What if we want to allocate the memory dynamically?

To dynamically allocate memory for the **Shape**, **Circle**, and **Rectangle** objects in your example, you can use the **new** operator.

Example program to achieve runtime polymorphism using virtual function

```
class Shape {
public:
    virtual void draw() {
        cout << "Drawing a shape" << endl;
    }
};

class Circle : public Shape {
public:
    void draw() {
        cout << "Drawing a circle" << endl;
    }
};

class Rectangle : public Shape {
public:
    void draw() {
        cout << "Drawing a rectangle" << endl;
    }
};
```

```
int main() {
    Shape* ptrShape;

    // Dynamically allocating memory for Circle
    ptrShape = new Circle();
    ptrShape->draw(); // Calls Circle's draw method
    delete ptrShape; // Deallocate memory

    // Dynamically allocating memory for Rectangle
    ptrShape = new Rectangle();
    ptrShape->draw(); // Calls Rectangle's draw
method
    delete ptrShape; // Deallocate memory

    return 0;
}
```


Pure Virtual Functions

A pure virtual function (or abstract function) is a special kind of virtual function that has **no body** at all.

A pure virtual function simply acts as a placeholder that is meant to be redefined by derived classes.

To create a pure virtual function, rather than define a body for the function, we simply assign the **function the value 0**.

Pure virtual function must be overridden by derived classes. If a derived class does not override the pure virtual function, it too becomes an abstract class.

Pure virtual functions allow defining a function interface in a base class without providing an implementation.

Pure Virtual Functions

A pure virtual function indicates that “it is up to the derived classes to implement this function”.

A pure virtual function is useful,

- when we have a function that we want to put in the base class, but only the derived classes know what it should return.
- we want our base class to provide a default implementation for a function, but still force any derived classes to provide their own implementation.

Abstract Class

- An **abstract** class is one that is **not** used to create **objects**.
- It is designed only to act as a **base** class.
- Abstract class cannot be **instantiated** but pointers and reference of abstract class type can be created.
- A class is **Abstract** class when it has **at least one pure virtual function**.

```

class Shape {
public:
    virtual void draw() = 0; // Pure virtual
    function
};

class Circle : public Shape {
public:
    void draw() {
        cout << "Drawing a circle" << endl;
    }
};

class Rectangle : public Shape {
public:
    void draw() {
        cout << "Drawing a rectangle" << endl;
    }
};

```

```

int main() {
    Circle circle;           // Circle object
    Rectangle rectangle;     // Rectangle object

    Shape* ptrShape; // pointer to base class
    Shape

    ptrShape = &circle;      // Pointing to
    Circle object
    ptrShape->draw(); // Calls Circle's draw
    method

    ptrShape = &rectangle;   // Pointing to
    Rectangle object
    ptrShape->draw(); // Calls Rectangle's draw
    method

    return 0;
}

```

Differentiate virtual base class and virtual function (April/May 2023)

Feature	Virtual base class	Virtual function
Purpose	To solve the diamond problem in multiple inheritance	To achieve runtime polymorphism
Usage	Used in class inheritance	Used in class member functions
Behaviour	Ensures that only one instance of the base class is created when multiple derived classes inherit from it	Allows derived classes to override the base class function
Syntax	<code>class Derived : virtual public Base {};</code>	<code>virtual void functionName();</code>

Try it!!!

Q. Write a program with **Student** as an abstract class and create derived classes such as **Engineering**, **Science**, and **Medical**. Show how objects of derived classes can be used through an array of base class pointers.
(December, 2018)

Hint:

Create objects of the derived classes: eg. `Engineering eng;`

Create array of base class pointers: `Student* students[3];`

Assign addresses of derived class objects to base class pointers: eg. `students[0] = ŋ`

```

// Abstract base class
class Student {
public:
    virtual void display() = 0; // Pure virtual function
};

class Engineering : public Student {
public:
    void display() {
        cout << "I am an Engineering student" << endl;
    }
};

class Science : public Student {
public:
    void display() {
        cout << "I am a Science student" << endl;
    }
};

class Medical : public Student {
public:
    void display() {
        cout << "I am a Medical student" << endl;
    }
};

```

```

int main() {
    // Create objects of derived classes
    Engineering eng;
    Science sci;
    Medical med;

    // Array of base class pointers
    Student* students[3];

    // Assign addresses of derived class objects
    to base class pointers
    students[0] = &eng;
    students[1] = &sci;
    students[2] = &med;

    // Loop through the array and call the
    display() method
    for (int i = 0; i < 3; i++) {
        students[i]->display();
    }
    return 0;
}

```



Any Queries?